

PR Assignment-2

Ques.1 States $\Rightarrow S_1 = \text{Sunny}, S_2 = \text{Rainy}$

Observations $\Rightarrow O_1 = \text{Happy}, O_2 = \text{Sad}$

Observation sequence = (Sad, Sad, Happy) = (O_2, O_2, O_1)

Initial probabilities $\Rightarrow \pi_1 = 0.6, \pi_2 = 0.4$

Transition matrix, $A = \begin{matrix} & \begin{matrix} \text{To} \\ \text{From} \end{matrix} & \begin{matrix} S_1 \\ S_2 \end{matrix} \\ \begin{matrix} S_1 \\ S_2 \end{matrix} & \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix} \end{matrix}$

Emission matrix, $B = \begin{matrix} & \begin{matrix} O_1 \\ O_2 \end{matrix} \\ \begin{matrix} S \\ R \end{matrix} & \begin{bmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{bmatrix} \end{matrix}$

① Forward algorithm

We compute: $P(O|\lambda) = \sum_i \alpha_T(i)$

Step 1: Initialization ($t=1, O_1 = \text{Sad}$)

$$\alpha_1(i) = \pi_i \cdot b_i(O_1)$$

$$\alpha_1(1) = 0.6 \times 0.2 = 0.12$$

$$\alpha_1(2) = 0.4 \times 0.6 = 0.24$$

Step 2: Recursion ($t=2, O_2 = \text{Sad}$)

$$\alpha_2(j) = \left[\sum_i \alpha_1(i) a_{ij} \right] b_j(O_2)$$

For S_1 :

$$\alpha_2(1) = [0.12 \times 0.7 + 0.24 \times 0.4] \times 0.2 = 0.036$$

For S_2 :

$$\alpha_2(2) = [0.12 \times 0.3 + 0.24 \times 0.6] \times 0.6 = 0.108$$

Step 3: Recursion ($t=3, O_3 = \text{Happy}$)

For S_1 :

$$\alpha_3(1) = [0.036 \times 0.7 + 0.108 \times 0.4] \times 0.8 = 0.05472$$

For S_2 :

$$\alpha_3(2) = [0.036 \times 0.3 + 0.108 \times 0.6] \times 0.4 = 0.03024$$

$$\text{Final likelihood: } P(O|A) = 0.05472 + 0.03024 = \boxed{0.08496}$$

⑥ Viterbi algorithm

$$\text{We compute: } \delta_t(j) = \max_i [\delta_{t-1}(i) a_{ij}] \cdot b_j(o_t)$$

Step 1: Initialization ($t=1$, $O_2 = \text{Sad}$)

$$\delta_1(1) = 0.12, \quad \delta_1(2) = 0.24$$

$$\delta_1(2) > \delta_1(1)$$

Step 2: $t=2$, $O_2 = \text{Sad}$

For S_1 :

$$\delta_2(1) = \max(0.12 \times 0.7, 0.24 \times 0.4) \times 0.2 = 0.0192 \quad (\text{from } S_2)$$

For S_2 :

$$\delta_2(2) = \max(0.12 \times 0.3, 0.24 \times 0.6) \times 0.6 = 0.0864 \quad (\text{from } S_2)$$

$$\delta_2(2) > \delta_2(1)$$

Step 3: $t=3$, $O_1 = \text{Happy}$

For S_1 :

$$\delta_3(1) = \max(0.0192 \times 0.7, 0.0864 \times 0.4) \times 0.8 = 0.027648 \quad (\text{from } S_2)$$

For S_2 :

$$\delta_3(2) = \max(0.0192 \times 0.3, 0.0864 \times 0.6) \times 0.4 = 0.020736 \quad (\text{from } S_2)$$

$$\delta_3(1) > \delta_3(2)$$

Final likelihood: $\boxed{0.027648}$ Path: Rainy $\xrightarrow{S_2}$ Rainy $\xrightarrow{S_2}$ Sunny $\xrightarrow{S_1}$

Ques. 2 Parzen Window Density Estimation method:

Parzen Window Estimation is a non-parametric method used to estimate the probability density function (PDF) of a random variable based on a finite set of data samples.

Basically, Parzen Window density estimation is an extension of the histogram. In a histogram, we count how many points fall into fixed bins. However, histograms are discontinuous and depend heavily on where the bin edges are placed.

The Parzen Window method solves this by placing a Kernel Function, $K(u)$, at every single data point. The total density at any point x is the average contribution of all kernels at that location.

→ For a dataset $\{x_1, x_2, \dots, x_n\}$ in d -dimensional space, the density estimate $\hat{f}(x)$ is defined as:

$$\hat{f}(x) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

where: n = total no. of samples

h = bandwidth (smoothing parameter)

$K(\cdot)$ = Kernel function

→ Derivation with a Gaussian Kernel

While the "window" can be a simple hypercube (the "Top-hat" Kernel), researchers often prefer the Gaussian Kernel because it

produces a smooth, differentiable density estimate.

Step 1: Define the Gaussian Kernel.

The standard d -dimensional Gaussian kernel is given by:

$$K(u) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2}\|u\|^2\right) \quad \text{--- ①}$$

Step 2: Substitute the distance variable

In the Parzen formula, our variable u represents the scaled distance between the observation x and the data point x_i :

$$u = \frac{x - x_i}{h} \quad \text{--- ②}$$

Substituting, Eqn. ② in Eqn. ①, we get,

$$K\left(\frac{x - x_i}{h}\right) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|x - x_i\|^2}{2h^2}\right)$$

Step 3: Combine into the Density Estimator:

Now, we plug this specific kernel back into the general Parzen window eqn:

$$\hat{f}(x) = \frac{1}{nh^d} \sum_{i=1}^n \left[\frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|x - x_i\|^2}{2h^2}\right) \right]$$

The final equation:

By simplifying the constants outside the summation, we arrive at the

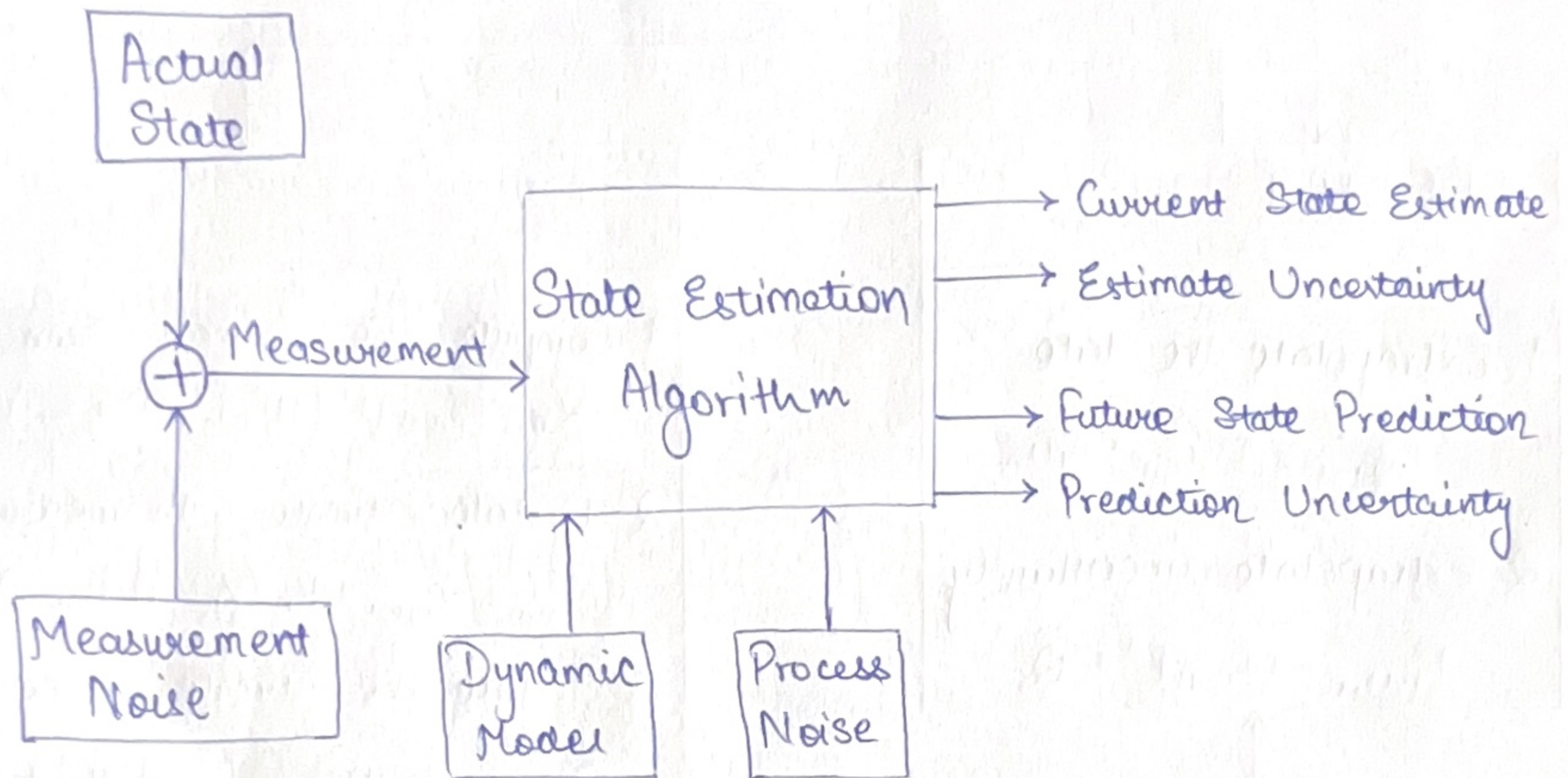
final form used in Kernel Density Estimation (KDE):

$$\hat{f}(x) = \frac{1}{n(2\pi h^2)^{d/2}} \sum_{i=1}^n \exp\left(-\frac{\|x - x_i\|^2}{2h^2}\right)$$

Ques. 3 Kalman Filter:

The Kalman Filter is an optimal recursive algorithm used to estimate the state of a dynamic state system from noisy measurements.

→ System Architecture:



* The Problem: we want to know the ACTUAL STATE, but we can only see a MEASUREMENT that is corrupted by MEASUREMENT NOISE.

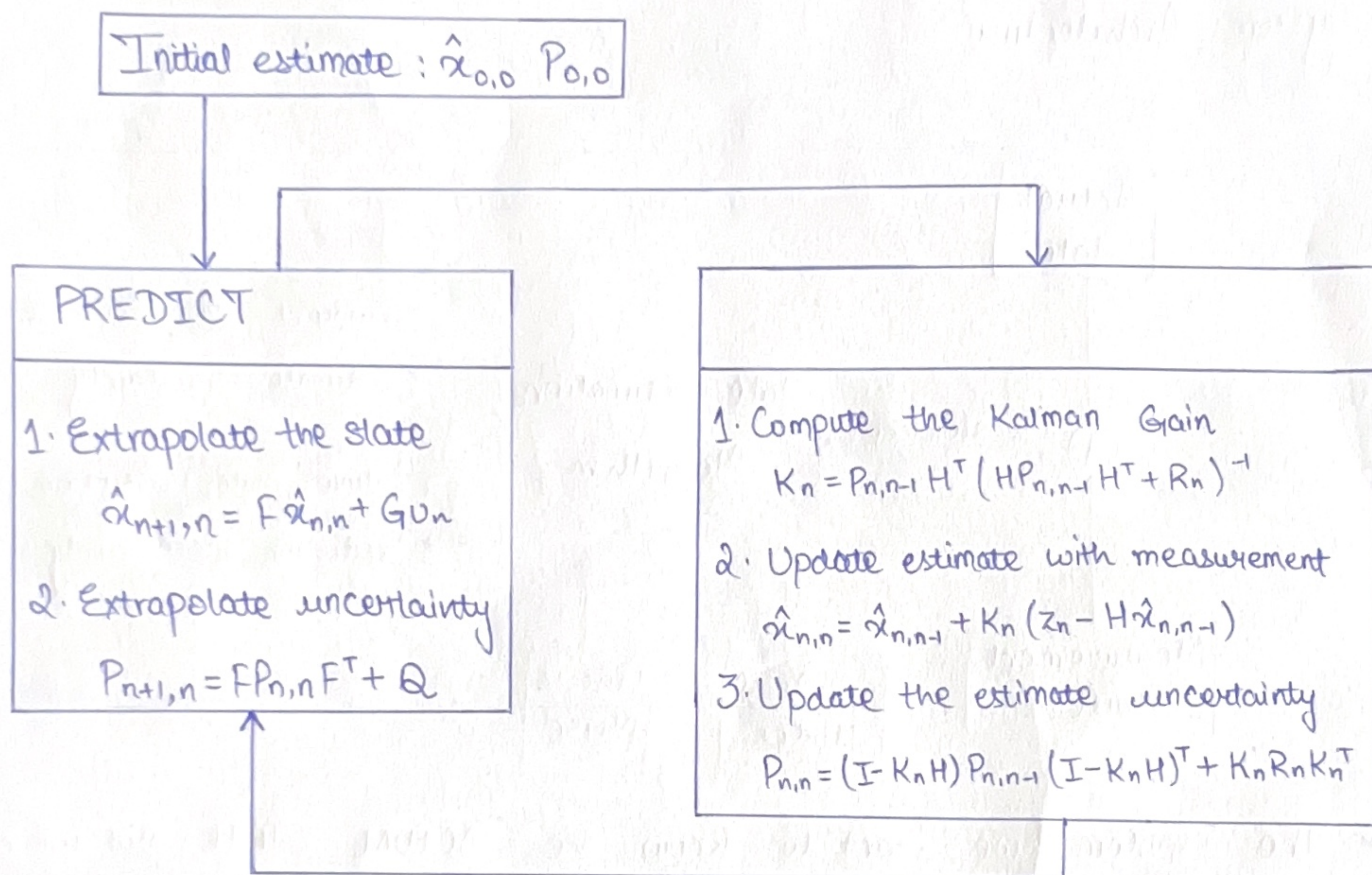
* The Internal logic: The STATE ESTIMATION ALGORITHM takes this noisy measurement and combines it with a DYNAMIC MODEL (the physics of the system).

* Accounting for Implementation: The algorithm acknowledges that the DYNAMIC MODEL isn't perfect by factoring in PROCESS NOISE.

* The Output: It produces two pairs of results:

→ Logic: Predict - Update loop

The Kalman filter follows a circular two-step process. The process can be thought of as a "refined guess" that gets corrected everytime new data arrives.



where, $\hat{x}_{n,n}$ = estimated system state vector at time step n

- $\hat{x}_{n+1,n}$ = predicted system state vector for time step $n+1$, computed using information
- u_n = control / input variable, representing known external time inputs to the system
- F = state transition matrix
- G = input/control matrix or input transition matrix, which maps i/p to state variables
- $P_{n,n}$ = covariance matrix (squared uncertainty) of the current state
- $P_{n+1,n}$ = covariance matrix (squared uncertainty) of the predicted state
- Q = process noise matrix